Frontend boilerplate — мастер-план

2026-10-15 · время чтения ~25 минут · ссылки вида [05b §6] ведут в `research/`; версии и обоснования по шаблонам — в [17] и `research/stack-report.pdf`

Коротко

- Строим репозиторий, из которого одной командой получается рабочий фронтенд-проект: pnpm-монорепо с выбранными шаблонами, общим UI-китом и токенами, настроенным Claude Code, проверками, Docker и CI на GitHub. Бэкенд внешний по OpenAPI, деплой на своих серверах, только веб.
- Шаблоны: `landing` и `blog` (Astro static), `site` (Next SSR, одно Next-приложение на проект; подшаблоны `site/base` — универсальный сайт с кабинетом, `site/shop` — база + commerce), `tma`, `admin`, `game` (Vite SPA; игра на DOM/CSS + dotLottie). Общее: React 19, Base UI kit, Tailwind через `tv()` в файлах-спутниках, MobX без DI + TanStack Query, плавные переходы экранов, все устройства и способы ввода, маппинг ошибок API ровно как в xRocket.
- AI-слой: корневой `CLAUDE.md` ≤ 200 строк, rules по путям, 21 skill (включая `/onboard`, `/errors-map`, `/speed`), 6 агентов, 4 хука, гейт `pnpm verify` и drift-тест `pnpm ai:check`; память только в репо; guard блокирует незапрошенный push; агент сам ведёт маппинг ошибок и их тексты через CLI (черновики на утверждение).
- Дизайн: шаг 1 — каталог компонентов в Storybook с утверждением пользователем; режимы Operate/Persuade/Read/ Experience, `DESIGN.md` на приложение, ~45 анти-слоп правил на lint; при готовом дизайне инициатива не поощряется.
- Локальный `apps/utils` (маппинг ошибок, тексты всех языков, каталог, экраны, токены) пишет файлы в репо и не коммитит. Init `pnpm run setup` вычитает невыбранное по манифесту, проверяет сборку, умеет dry-run и откат, в конце зовёт `/onboard`; pixel-perfect по Figma — модуль со снимком макета в репо.
- Три волны: ядро + tma + admin + AI + init + utils → landing + blog + site/base + game + anti-slop → site/shop + опции site + Figma-модуль + CI-ревьюер + итоговый README; roadmap: canvas-игры на PixiJS и новые подшаблоны site. Открытых вопросов к пользователю нет (§12).

---

1. Что это

Слой	Содержимое	Откуда
Шаблоны `apps/*`	6 шаблонов, любое подмножество; `site` с подшаблоном `base` или `shop`	[05], [14], [16]
Ядро `packages/*`	`ui`, `tokens`, `api` (клиент, маппинг ошибок, мост Query↔MobX), `i18n`, `config`, `telegram-platform`, `lottie` (обязателен для game, у остальных по опции), `realtime`, `analytics`, `astro-shared`, `devtools-core` (логика utils и CLI)	[01 R1], [05 §5], [14 R3], [16], [19]
Utils	`apps/utils`: локальный dev-сайт проекта (маппинг ошибок, тексты, каталог, экраны, токены); не деплоится по умолчанию	[19]
Правила	архитектура, код, компоненты, дизайн, безопасность; каждое с проверкой или пометкой `manual`	[03], [04], [07], [10], [11]
AI-слой	дерево `CLAUDE.md`, rules, skills, agents, hooks, settings, гейты	[08], [15]
Init	выбор, вычитание по манифесту, проверка, откат, `/onboard`	[06]
Инфраструктура	Dockerfile на приложение, общие nginx/security snippets, compose, GitHub Actions, runbooks	[05], [10], [11], [16 §6]
Модули	pixel-perfect по Figma, anti-slop enforcement, аналитика с consent	[07], [09]

Сознательно нет: Nx, RN-стека, tsyringe и декораторов, Redux, второго генератора API, CRUD-фреймворков, UI-библиотек со своим дизайн-языком, canvas-движка в игре, своего auth-сервера, CMS, WebSocket-прокси через Next, Vercel-специфики, Strapi как источника маппинга, своей галереи компонентов [02], [05], [14], [16 §1.3], [19].

Принципы (правила пользователя): каждый шаблон работает на macOS, Windows, iOS и Android, с касанием, мышью и клавиатурой, на любом вьюпорте; каждый шаблон имеет плавные переходы между экранами с учётом `prefers-`

`reduced-motion`, проверенные в iOS и Android WebView и на десктопе; при готовом дизайне агент реализует макет без собственных визуальных идей; у каждого проекта крупнее лендинга есть каталог компонентов, и он строится первым.

Критерий результата: после `git clone`, `pnpm run setup` и `/onboard` любой агент на любой машине работает по одним правилам, проект собирается и запускается, а человек без ИИ проходит «симптом → файл → правка → выкладка» за 30 минут [11 R1].

---

2. Шаблоны и опции

id	Для чего	Рендеринг / рантайм	Чем жертвует
`landing`	промо, маркетинг	Astro static, React-острова, nginx; опция `landing-node`	клиентское состояние, общий React-контекст между островами [05a]
`blog`	статьи, авторы, RSS; опция `docs`	Astro static; общий слой в `packages/astro-shared`	то же
`site`	`base`: универсальный SSR-сайт с кабинетом (курсы, видеохостинг, биржа); `shop`: база + commerce (каталог, фасетные фильтры с URL-состоянием, поиск, корзина, чекаут через провайдера)	Next App Router + Cache Components, Node standalone; ровно одно Next-приложение на проект; образ на окружение (ключ Server Actions вшивается при сборке)	вес рантайма Next, сложность модели кэша, nonce-CSP только в динамической зоне [16], [17]
`tma`	Telegram Mini App, Stars, TON Connect (опция)	Vite SPA, nginx	SSR, SEO, PWA (iOS WKWebView без SW) [05c]
`admin`	админка, дашборд, SaaS-кабинет	Vite SPA, nginx; опция `bff`	SSR, тонкая оптимизация бандла [05d]
`game`	казуальные механики: колесо, слоты/кейсы, карты, мины; браузер и TMA (своё приложение)	Vite SPA + React + kit, DOM/CSS/WAAPI + Motion, dotLottie в Worker, видео-пререндер	SSR, SEO; сцены с сотнями объектов (будущий `game-canvas`) [14]

Опции init (удаляются с кодом, пакетами, доками и skills): `content-cms`, `docs`, `landing-node`, `gsap` / `lenis` (только landing), `pwa` (не для tma), `ton-connect`, `tg-analytics`, `bff` (admin), `analytics` (Umami по умолчанию / GA4 / Метрика / PostHog с consent), `figma` (§7), `lottie` (вне game), `utils` и `catalog` (вкл., если выбран любой шаблон кроме landing/blog), `utils-deploy` (поздняя опция, выкл.), `crowdin` (выкл.); для site: `auth-bff` (вкл.), `i18n-routing` (вкл.), `realtime`, `video`, `multi-instance`; подшаблон `site/shop` добавляет домен commerce, `search`, `commerce-shopify` и требует `auth-bff` [16 §5.3]; для game: примеры `wheel` + `reel` + `cards`, `lottie-video`, `realtime` [14]. Граф пакетов: `game` или `lottie` → `packages/lottie`; `tma` или `game` → `telegram-platform`; `realtime` → `packages/realtime`; `utils` → `devtools-core`. Один выбранный шаблон всё равно остаётся монорепо.

---

3. Технологии

3.1 Общее ядро

Слой	Решение	Источник
Язык, Node, pnpm	TS 6; Node 24 LTS (26 одним PR в Q1 2027); pnpm 11 с `catalog:` на последний патч старше 7 дней	[17 §1], [10 §1.6]
Сборка, lint	Turborepo; Vite у пяти шаблонов, Turbopack у site; ESLint 10 с `@eslint-react` и `jsx-a11y-x`; Prettier, stylelint, knip	[17 §1]
UI kit	`packages/ui`: shadcn-подход (код в репо) на Base UI; `ModalPresence` (exit-анимации, возврат фокуса); `overflow: clip`, модалки порталом; касание, мышь и клавиатура у всех компонентов; `FullScreenResult`, `Toast`, `PageQueryGuard` для слоя ошибок	[05b §4], [15 §C], [18 §5.5]

Слой	Решение	Источник
Переходы экранов	один API `ScreenTransition`: View Transitions там, где поддерживаются, иначе CSS-fade 200–240 мс, reduced-motion → мгновенно; Astro — кросс-документные VT, site — React `<ViewTransition>`, SPA — `defaultViewTransition` TanStack Router; real-time никогда через `startTransition`; длительности из токенов; минимальная iOS 16.4	требование пользователя; [17 §2], [16 §4]
Стили, токены	Tailwind v4, классы только через `tv()` в `Name.styled.ts`; в `.tsx` только `className={button({ tone, size })}`; тема сброшена до токенов; fallback CSS Modules + `data-*`. `packages/tokens` → `tokens.css`, `@theme inline`, `canvas.ts`; примитивы → роли → компонентные; hover-токены под `(hover: hover)`; в TMA `--tg-theme-*` мапятся на роли, брендовый слой включается флагом	[13], [03 U1], [15 §C]
Состояние	TanStack Query + MobX plain-классы, без DI/декораторов, без Zustand/Redux; свой мост `MobxQuery v2` обязателен; в site RSC-prefetch + `HydrationBoundary`, MobX только в клиентских компонентах; в Astro MobX нет; React Compiler выключен	решение пользователя; [01 §4], [17], [18]
API и маппинг ошибок	`openapi-typescript` + `openapi-fetch` (нужен `schemaPath` в middleware; при остановке поддержки вендорим в `packages/api`); свой генератор фабрик query/mutation из спеки + JSON маппинга; `@medusajs/js-sdk` обёрнут в тот же маппинг. Маппинг — данные `packages/api/src/errors/mappings/<svc>.json` + JSON Schema, правятся в utils или CLI, компилируются в TS; тексты в namespace `apiErrors`	решение оркестратора по [18 §5.1] (заменяет Orval из [17])
Анимации, формы, таблицы	CSS/WAAPI на `transform` / `opacity`, Motion только вне kit, dotLottie через `packages/lottie`; RHF + Zod (запрет `watch()`); TanStack Table + Virtual; Recharts	[14 F3], [05 §5]
Роутинг, i18n	SPA: TanStack Router, virtual file routes с `routes.ts` на домен; site: App Router + nuqs; i18next везде (site `next-i18next`, Astro только на сервере); каталоги в формате xRocket `translations/{all,<locale>}/<ns>.json` с `sync`, валидаторами и `i18n:check` в `packages/i18n/tools`; Crowdin опцией поверх `en/*`	[architecture §1], [17 §2], [18 §5.8]
Real-time	`packages/realtime`: WebSocket или SSE, reconnect, heartbeat, rAF-батч, `useRealtimeChannel()` → MobX или `setQueryData`; одноразовый тикет из BFF, сокет напрямую к бэкенду	[16 §1.1]
Telegram, диагностика	`packages/telegram-platform`: единственное место импорта SDK, mock, нижний inset поднимает весь контент, демо-режим без Telegram, `pnpm tunnel` к прод-сборке; панель `?perf=1` / `startapp=perf` в tma и game (путь рендера, причина отката, fps, CSP)	[17 §2], [15]
Каталог, тесты, Docker	один Storybook 10 в `packages/ui` (`addon-vitest`, `addon-a11y`, `addon-mcp`; stories приложений тем же glob), стартует вместе с utils; Vitest, MSW, Playwright (скриншоты только в Docker), Lighthouse CI; перф и e2e на прод-сборке; матрица Chromium + WebKit × 390/430/1024/1440, касание и клавиатура в e2e, ручной чек-лист на устройствах; Sentry lazy, `log.*` вместо `console`. Dockerfile на приложение, `compose.yaml` генерирует init; build `node:24-slim`, runtime `nginx-unprivileged` или Node, по digest; GitHub Actions	[19 §5], [15 §6], [11 R7], [06 F3], [17 §1]

3.2 По шаблонам

Подробный стек — в `stack-report.pdf` [17]. Что расходится: landing/blog — build-time данные, без стора и API-слоя (форма с API берёт `packages/api`); site — RSC через `domains/<d>/api.server.ts` + `'use cache'` + теги для публичного, Query с гидрацией для интерактива, BFF-сессия `__Host-` + DAL `getSession()`, `[locale]`; site/shop — `CommerceProvider` (`mock` / `medusa` / `shopify`) с вебхуком ревалидации, фасеты в URL, `PaymentStep` (ЮKassa redirect, Stripe Payment Element со строгим CSP чекаута); tma — `initData` → сессия, тема Telegram + брендовый слой, BackButton ↔ роутер; admin — data-table с URL-состоянием, OIDC PKCE за своим адаптером; game — `logic/` чистый TS → MobX VM + очередь презентации → UI, исход всегда от сервера, Lottie-конвейер (Worker, видео-пререндер, `play()` в тапе), тест CSP-откатов. Правила kit для трёх сред (RSC, острова, SPA), на lint [05 §7]: нет импортов `next/*`, `astro:*`, SDK; `'use client'` пофайлово; нет `window` на уровне модуля; `Link` / `Image` через `render`-проп; нет обязательных провайдеров; анимации kit на CSS.

3.3 Что берём из xRocket и реальных игр, что нет

Берём: доменные вертикали, тонкая страница + VM; весь pipeline ошибок (§4); синк Swagger и UI редактора из xrocket-tools; формат каталогов `all/`; «баг → ESLint-правило», ratchet-тесты, runtime env, visual-тесты в Docker; из игр — лента на `translate3d` + CSS `linear()`, провабли-фэйр, Worker-конвейер Lottie, `?perf=1` [01]–[04], [14 F2], [15], [18],

[19]. Адаптируем: VM только для экранов с оркестрацией; маппинг как данные в репо, тексты в i18n; снимок макета вместо MCP; очередь презентации вместо `setTimeout`. Не берём: RN-стек, tsyringe, Redux, Nx, werf, `eslint-disable` пачками, 61 КБ CLAUDE.md, memory как хранилище правил, Strapi как источник маппинга, react-spring по `left/top`, canvas-движок в базе, страницу на 8778 строк [14 F1–F2].

---

4. Архитектура и правила кода

Дерево приложения (одно для всех; Astro и Next хранят маршруты по правилам фреймворка, домены те же):

```
apps/<id>/src/
  app/          composition root: провайдеры, init-пайплайн, router.ts, observability, ErrorBoundary,
FailureScreen
  domains/<d>/  вертикаль: routes.ts, pages/<Page>/ (<Page>.tsx + model/ + компоненты плоско), components/,
api.ts, index.ts
  shared/       без бизнес-знаний: lib, config (env, flags), i18n-каталоги (all/ + языки), model
(глобальные сторы)
  logic/        только game: чистый TS без React/MobX/DOM (ESLint), тесты в Node
apps/utils      Hono на 127.0.0.1 + Vite SPA на kit (Errors, Texts, Catalog, Screens, Tokens)
packages/       ui  tokens  api (specs, generated, errors/mappings/*.json, middleware, query)  i18n  config
telegram-platform  lottie  realtime  analytics  astro-shared  devtools-core
tools/          checks/ (один раннер проверок), init/, eslint-rules/, figma/
```

В `site` маршруты Next лежат в `src/app/` и тонкие (≤ 60 строк, без `'use client'`); в домене `api.server.ts` (`server-only`), `queries.ts`, `actions.ts`; подшаблон `shop` добавляет `domains/commerce/`, группу `(shop)` и `/api/revalidate`, которые init удаляет при `site/base` [16 §5.1].

Границы (арх-правило в `tools/checks`, в гейте и CI) [01 R2]: приложения не импортируют друг друга; пакеты только через `exports`; `ui` и `tokens` не знают сторов, `api`, приложений; `shared` не импортирует `domains` и `app`; чужой домен только через его `index.ts`; относительные `../` между слайсами запрещены; сгенерированный API и маппинги только через барел `@repo/api` (барел первым подключает middleware); `index.ts` только у домена и пакета, без `export *`; глубина от `src/` ≤ 6 [11 R5]. Пакет при втором потребителе, абстракция при третьем [11 R6.4].

Состояние и VM: серверные данные только через `MobxQuery v2` (`QueryObserver` → observable); глобальные сторы создаются в `app/` и передаются React-контекстом; VM создаётся явно в `useViewModel(() => new XViewModel({ ... }))`; store тонкий, VM богатая, `pageState` как union состояний [03 U20]. В SSR-шаблонах сторы и VM только в клиентских компонентах, создаются на запрос. VM не навигирует и не зовёт SDK: handler возвращает результат, навигирует владелец кнопки. Покадровые значения живут вне React и MobX; серверные события игры идут в очередь презентации [14 R5–R6].

Ошибки API ровно как в xRocket (требование пользователя; спецификация [18 §5]): везде, где фронт зовёт бэкенд, стоит цепочка middleware `openapi-fetch` `[appStatusGate, auth, maintenance, responseResolution, bannedGuard, retry, unauthorized]` (порядок закреплён тестом). `responseResolution` ищет запись по `(schemaPath, METHOD_STATUS_type)`: точный вариант → для 2xx любой `METHOD_2xx` → `METHOD_ERR` → `null`; `toast` показывает middleware (Support с кодом при `_ERR` и ≥ 500), `modal` кладёт `_resolution` в тело и рисует `PageQueryGuard` на page root, `custom` разбирает домен, `silent` молчит; успехи тоже задаются данными. `retry` только GET и ≥ 500; 401 чистит сессию только по текущему токену. `appStatus` — computed в `AppStore`: `initial | app | no-network | server-error | banned | maintenance`; при любом статусе кроме `app` / `initial` роутер не смонтирован, рендерится только неубираемый `FailureScreen` (отдельная группа маршрутов), `appStatusGate` отвечает 503 на все запросы. Исправления доказанных поломок xRocket: `*_NET`-тосты через `onError`; статус `maintenance` со своим экраном (маркер 503 + problem type, настраивается); `no-network` восстанавливается по `online` + health-ping; на SSR тосты per-request; импорт только через барел (lint). В site тот же маппинг работает в серверном fetch-слое: серверные ошибки идут в error boundary или `not-found`, клиентские — в тосты и модалки. Правило в API-rule (требование пользователя): новый или изменённый эндпоинт не готов, пока нет его маппинга и текстов; агент добавляет их сам через CLI и `/errors-map` (§5), гейт проверяет `ERR`, `NET` и 2xx у каждой операции и каждый ключ текста на всех локалях.

Файлы компонента (решение пользователя, на lint): `Button/Button.tsx` + `Button.styled.ts` (все `tv()`-рецепты) + `types.ts` + `consts.ts` + `utils.ts` + `index.ts` + stories в kit [13 R2]. Типы, UPPER_CASE-константы, строковый `className` и `style` без CSS-переменной в `.tsx` запрещены; kit закрыт для внешнего `className`. Один компонент на

файл; `.tsx` ≤ 200 строк, любой файл ≤ 700 (error); подкомпоненты плоско рядом; имя файла = имя экспорта. Комментарии: только там, где код без них не понять, и они описывают, как работает механизм. Деньги только через decimal.

Принуждение: ESLint-набор из tooling.md §4 без RN-правил; MobX-правила xRocket (`no-store-in-page-view` на `pages/**` с лимитом отключений ≤ 10, guard только в page root, ручная мутация с `mutationKey`); для анимаций только `transform` / `opacity`, запрет `left/top`, `setTimeout` для синхронизации и импортов `framer-motion` / `lottie-web` [14 R12]; для site: `'use cache'` только в `api.server.ts` с `cacheTag`, `getSession()` в каждой защищённой странице [16 §5.2]; knip; i18n-проверки. Правило без проверки помечено `manual`.

---

5. AI-практики

Дерево инструкций [08 §1–2]: корневой `CLAUDE.md` ≤ 200 строк (session start по выводу хука и ссылка на `/onboard`, карта «перед чем читать → что грузится само» с utils и его CLI, команды, 25–30 правил одной строкой с механизмом, делегирование, формат отчёта); канон в `.claude/rules/*.md` с `paths:`; `apps/<id>/CLAUDE.md` ≤ 150 строк (у site — секция подшаблона shop с маркером init) и `packages/<pkg>/CLAUDE.md` ≤ 80. Шаблонных rule-файлов нет: всё про шаблон удаляется с его папкой. AI-доки на английском; ответы пользователю на русском. Codex не поддерживается [08 §2.5].

Skills (21 core [08 §4.1]): процесс (от `plan-task` до `push` и `record-decision`), генерация (`new-page`, `new-component`), дизайн и Figma, `setup-check`, `qa-handoff`, `init-project`; `onboard` (требование пользователя): после init агент читает весь контекст (CLAUDE.md-дерево, rules, README приложений и пакетов, `ARCHITECTURE.md`, `DESIGN.md`, ADR, `.boilerplate.json`), строит короткую сводку в сессии (стек, команды, правила, карта, utils и CLI, deliberate-решения) и дальше пишет код ровно как задокументировано; read-only, идемпотентен, упомянут в session start. `errors-map <svc>` (требование пользователя): найти операции без маппинга → предложить type (`toast` / `modal` / `silent` / `custom`) и severity на каждый вариант → записать маппинг и тексты `apiErrors` на всех локалях → прогнать проверки → показать diff. Это единственное исключение из правила «тексты не придумывать» и только для ошибок: тексты агента помечены `"draft": true` (бейдж в Errors и Texts), перечислены в отчёте на утверждение, тексты пользователя всегда побеждают. В UI utils агент не ходит: те же операции через CLI с `--json` (`pnpm errors:add|set|list|sync|check`, `pnpm i18n:set|missing|check`) на одной с utils библиотеке валидации (схема, плейсхолдеры, плюрали, все локали); компоненты ищет через Storybook MCP до создания нового. Опциональный `orchestrate` выключен по умолчанию [15 §E]. Шаблонные skills с префиксом `<id>-` удаляются манифестом.

Агенты и режимы скорости [20 §1]: `researcher`, `implementer`, `code-reviewer`, `rules-reviewer`, `design-reviewer`, `figma-section-inspector`; модель и effort берутся из активного режима, который ставит skill `speed` (пишет `.work/mode` и строку в контекст сессии): `comfort` (по умолчанию) — Opus 5.5, effort medium, Fable только для оркестрации и планирования, вложенность ≤ 2 (главный агент запускает сколько нужно, каждый его агент не больше одного); `econom` — Opus 5.5, без Fable, вложенность 1, не более 5 субагентов, планка качества та же; `business` — без лимитов, Opus 5.5 на effort `max`, Fable используется часто, приоритет скорость, качество, глубина. Параллельные агенты работают каждый в своём git worktree со своим `pnpm install`, без `git stash` / `reset` на общей копии; сборка для устройства только из закоммиченного состояния; ассет агента помечается для дизайнера [15 §E].

Settings и хуки [08 §3, §5], [10 §4.3]: output style Human, auto memory выключена, `acceptEdits`; deny на чтение секретов и разрушительные git/shell; ask на commit, установку пакетов, сеть. **Push**: текстовые запреты не сработали (три незапрошенных push в сессии Шахтёра [15 §7]), поэтому guard блокирует `git push` без одноразового маркера, который ставит только пользовательский skill `/push`. MCP: Playwright и Storybook; Figma MCP как опция модуля. Хуки на Node: SessionStart печатает `[session-check v1]` и активный режим; PreToolUse `guard.mjs` блокирует секреты, разрушительные команды, защищённые пути, push без маркера, неверный формат коммита; PostToolUse форматирует и линтит тронутый файл; Stop ловит остатки. Тесты хуков в гейте.

Гейты: `pnpm verify` = typecheck + lint + tests затронутых проектов + i18n + `errors:check` (JSON по схеме, у каждой операции `ERR`, `NET` и 2xx, каждый ключ текста на всех локалях, `{{param}}` совпадают) + `catalog:coverage` (stories и Matrix у каждого экспортируемого компонента) + knip + `tools/checks` + тесты хуков; `--all` в CI; `--hotfix` = typecheck + unit + build. Pre-commit = lint-staged + secretlint + i18n `sync` + маркеры. `pnpm ai:check` (13 drift-проверок [08 §5.2]: пути и команды, валидность skills и rules, «enforced by» реально включены, язык AI-доков, маркеры init). Правило: «fix the doc, not the test».

Память, планы, процесс [08 §6]: ничего не живёт только в memory; «запомни» → `/record-decision` пишет в rule, дочерний CLAUDE.md или ADR и показывает diff; планы `.work/plans/` в git, агент их не коммитит. Процесс: session check → контекст → план со всеми ⚠️ одним сообщением → approval → реализация по пунктам с гейтом → `fresh-review` двумя свежими агентами → отчёт → ожидание. Коммит только по прямой просьбе.

---

6. Дизайн-система и каталог

Шаг 1 любой дизайн-работы — каталог компонентов (требование пользователя; и при готовом дизайне, и без него) [19 §7]: инвентарь из DESIGN.md или снимка Figma, поиск существующих через Storybook MCP → для каждого нового или расширенного компонента stories, где обязательна **Matrix**: все варианты × размеры × состояния с подписями, RTL-экземпляр, реалистичные данные, обе темы и узкая ширина → `addon-vitest` + a11y и VRT в Docker → пользователь смотрит каталог (вкладка Catalog в utils: showcase-обзор всех Matrix с переключателями темы, локали и ширины) и утверждает → только потом экраны, и только из каталога; новый визуальный элемент при вёрстке экрана = возврат к шагу

1. Showcase-маршрутов внутри продуктовых приложений нет (проверка `tools/checks`); `catalog:coverage` в гейте.

Режимы [07 F4]: Operate (admin, tma, кабинет и рабочие экраны site, чекаут): привычность важнее выразительности, 0 входных анимаций. Persuade (landing, публичные страницы site, PDP): одна смелая вещь, остальное тихо. Read (blog, docs): ритм чтения, мера 65–75ch. Experience (game): UI из материала мира игры на компонентах kit [15 §C].

Процесс без дизайна [07 F7]: `apps/<id>/DESIGN.md` до визуальной работы (режим по зонам, референсы и антиреференсы, палитра 4–6 ролей в OKLCH обеих тем, шрифты, шкалы, движение, «одна запоминающаяся вещь», copy voice, Exceptions с причиной); токены приложения пишутся из DESIGN.md; после кода матрица скриншотов 2 темы × 3 ширины, сканы, свежий `design-reviewer`, не больше двух раундов. **При готовом дизайне инициатива не поощряется**: агент реализует макет без собственных визуальных идей; анти-слоп lint остаётся полом для всех.

Жёсткие правила (H1–H45 в [07], шаблонные в [07a]): нет градиентного текста, декоративных градиентов, свечений, иконок в цветных плитках; один акцент + нейтраль + статусы; стоп-палитры и reflex-шрифты только с записью в DESIGN.md; тени только у плавающих слоёв; нет eyebrow-капса; кегли из шкалы (минимум 12px, поле 16px на мобиле); нет карточки в карточке, шаблонного hero, fade-up у каждой секции; hover только яркость/фон; 100–300 мс ease-out без bounce, `transition: all` и бесконечных анимаций; `prefers-reduced-motion` обязателен; одна библиотека иконок, нет ✨, em dash, `A · B · C` и `→` в кнопках; заголовок существительное, кнопка с глагола; нет придуманных цифр; пустое = что пусто + действие; цели ≥ 40×40 (44 для tma/game); фокус-кольцо 2px; реальные `h1–h3`.

Enforcement (Tailwind v4 + `tv()` [13]; всё на `error`, `warn` = долг [03]): токены и файл-спутник — сброс темы в `@theme`, `better-tailwindcss` (`no-unknown-classes`, запрет произвольных `[..]`), ESLint (`className` только из рецепта, `style` только с CSS-переменной, `tv`/`clsx` только в `*.styled.ts`, там запрет hex/rgb/oklch), stylelint [13 R4]; запрещённые классы и шрифты — `no-restricted-classes` по списку [07 F8], `--font-*: initial`; текст — `lint:copy` по i18n-JSON; структура, палитра, контраст — `design:scan` (Playwright, DOM-эвристики; сначала отчёт, затем gate) и скрипт по `tokens.css` (ΔE к стоп-цветам, контраст ролей); CI требует `DESIGN.md`.

---

7. Pixel-perfect по Figma (модуль `figma`)

Ограничение: доступ к макетам обычно view-only, платных seats скорее всего нет; на бесплатном плане MCP и REST дают 6–20 вызовов в месяц [09 F2, F4]. Поэтому единственный режим — **снимок макета в репозитории**.

#	Этап	Механизм	Гейт
P0	контракт	`docs/design-qa-contract.md`: брейкпоинты, допуски, иерархия источников, что не бывает pixel-perfect [09 R10]	подписан до вёрстки
P1	экспорт	минимальный Figma-плагин «Export frame → JSON + PNG» (узлы, геометрия, стили, переменные → DTCG); запускает пользователь после запроса edit-доступа; fallback REST bulk и ручной ввод из Dev Mode	`design/specs/*.json` + `design/frames/*.png` в репо
P2–P3	токены, компоненты	DTCG → Style Dictionary → `tokens.css`, `canvas.ts`, `tokens:check` в CI; `design/component-map.json` (компонент Figma → kit + variant→props) и каталог из §6	PR с diff токенов; каждый инстанс в карте или ⚠️

#	Этап	Механизм	Гейт
P4	план	`/figma-implement` по снимку: Name map с `data-figma`, Reuse map, Token mapping (нет токена → ⚠️, ближайший молча не подставлять), i18n-ключи, Responsive table	approval по всем ⚠️
P5–P6	вёрстка и сверка	блок за блоком по таблице Figma → CSS [09 F3]; `figma:measure` (DOM-замеры в Docker по `data-figma`) блокирующий, кап 5 итераций на блок; `figma:diff` против PNG advisory; агенты-инспекторы по секциям	0 fail вне `design/deviations.md`
P7	приёмка	два свежих ревьюера, HTML-отчёт с overlay, запись отклонений, эталоны VRT из Docker	sign-off

Допуски [09 R7]: геометрия 0 (±0.5), позиция и текст ±1px, шрифт точно, цвет по токену, тени/блюр advisory; юзабилити выше пикселя. Не бывает pixel-perfect (в контракт): растеризация текста, переносы на реальных данных, нативные контролы, blur, дробные значения.

---

8. Init, utils и очистка

Модель — вычитание по манифесту [06]: исходный репо — рабочий монорепо со всеми шаблонами под обычным CI; init только удаляет и перегенерирует несколько файлов. У каждого файла ровно один владелец: core, модуль (шаблон, подшаблон или опция), граф пакетов или генератор. Шаблон = `apps/<id>` целиком + `docs/templates/<id>.md` + `.claude/skills/<id>-*`; подшаблон `site/shop` — папки commerce внутри `apps/site`, его секция CLAUDE.md и skills. Общие пакеты обрезаются по графу `workspace:*`; CI находит приложения по `apps/*/deploy.json`. Маркеры `<!-- init:<id> -->` только в Markdown; генерируются `compose.yaml` и таблица README. `apps/utils` без `deploy.json`: `tools/checks` следит, что он не попал в compose и CI-деплой без `utils-deploy`.

Интерактивный ход `pnpm run setup` (закоммиченный Node-бандл, работает до `pnpm install`; кириллица в пути учтена): preflight → имя проекта и npm scope → мультивыбор шаблонов → подшаблон для `site` → опции каждого выбранного шаблона → общие опции (аналитика, `figma`, локали, CI-хост; utils и Storybook включены автоматически, если выбран любой шаблон кроме landing/blog) → сводка плана → `--dry-run` или подтверждение → шаги с прогрессом (бэкап в `.init-backup/`, правки конфигов, install, обновление зависимостей до последнего патча старше 7 дней) → `pnpm verify --all` + поиск остатков → итог: `.boilerplate.json`, следующие шаги (`pnpm dev --filter <id>`, `pnpm utils`, `/onboard` в первой сессии Claude, предложенная команда коммита; init никогда не коммитит). Init и манифест остаются (`--purge-init` убирает), поэтому `--remove <id>` работает позже. При ошибке автооткат, при обрыве `--resume` / `--rollback`; повторный запуск идемпотентен; все шаги доступны флагами и `--json` для агента, skill `/init-project` ведёт его через список → вопрос пользователю → dry-run → применение → отчёт.

Utils [19 §6]: `pnpm utils` поднимает Hono-сервер на `127.0.0.1` с одноразовым токеном, Vite SPA на kit и Storybook одной командой. Вкладки: Errors (дерево сервис → операция → ответ, фильтры `new/modified/deleted`, «нет текста» и `draft`, правка type/severity/support, тексты всех языков рядом с превью настоящих `Toast` / `Modal`, «Sync with OpenAPI» по закоммиченной спеке — логика синка из xrocket-tools), Texts (приложение → namespace → ключ, колонки языков, missing, неиспользуемые и бейдж `draft`, плюрали, валидация `{{var}}` и тегов), Catalog (встроенный Storybook), Screens (VRT-снимки, diff с baseline), Tokens (роли и примитивы в обеих темах, контраст). Поток: UI → атомарная запись JSON в репо → `split` + валидаторы → `git diff` и «Run checks»; коммит и push только обычным путём. Тексты правит пользователь; агент добавляет только черновики ошибок через CLI (§5), пользователь утверждает их здесь. Логика в `packages/devtools-core`, та же у UI, CLI и pre-commit.

Тесты [06 F9]: на каждый PR статические проверки и юнит-тесты плана; e2e-матрица: на PR одиночные шаблоны (site в обоих подшаблонах), nightly pairwise + Windows с кириллическим путём и macOS; тесты dry-run, отказа на каждом шаге, идемпотентности и `--resume`.

---

9. Безопасность и бэкапы

База [10 §1]: фронт не хранит секретов; zod-схема env, серверная не импортируется в клиент; HTML и JSON-in-script только через `SafeHtml` и `serializeJsonForScript`; ESLint security-набор; заголовки из одного `security-headers.conf` и тот же набор в SSR middleware с e2e-тестом; CSP всегда сначала Report-Only; профиль game: `worker-src 'self'`

(через `setWorkerUrl`), `media-src 'self' blob:`, `wasm-unsafe-eval`, e2e на молчаливые откаты [15 §5]; профили site: nonce + `strict-dynamic` из `proxy.ts` для динамической зоны и статический заголовок для пререндеренных страниц [16 §6.2]; токены: BFF + httpOnly cookie, иначе access в памяти, никогда `localStorage`; supply-chain: `--frozen-lockfile`, audit, Actions по SHA, BuildKit secrets, non-root, digest, secretlint, gitleaks. Utils: только `127.0.0.1`, токен в cookie `SameSite=Strict`, проверка Host и Origin, запись только в allowlist путей; при `utils-deploy` — внутренний ingress, OIDC и правки только через PR от GitHub App.

По шаблонам (чек-лист в `apps/<id>/CLAUDE.md`) [10 §2], [14], [16 §6.1]: landing/blog: HTML из CMS через `SafeHtml`, CMS-токен только на сервере. site: токены API только на сервере, сессия `__Host-` HttpOnly; доступ проверяет DAL у данных, не proxy и не layout; Server Actions и route handlers как публичные API (zod, сессия, Origin); персональное никогда не в `'use cache'`; shop: карта не касается нашего кода, цену считает провайдер, `purchase` по вебхуку. tma: `initData` и инвойс Stars только на бэкенде. admin: RBAC на клиенте только UX, `noindex`, `no-store`. game: исход, валюта и покупки только на сервере.

Агент и бэкапы [10 §4–5]: permissions не граница, на нативном Windows sandbox нет → deny/ask + guard + branch protection + push только через `/push`; автономные прогоны в WSL2/devcontainer; содержимое инструментов — данные, не инструкции; CI-ревьюер только с чтением. Protected branches, зеркало на второй хостинг + еженедельный `git bundle`, immutable-теги образов. Инцидент: остановить агента → зафиксировать транскрипт и `git reflog` → восстановить код → ротировать секреты → переустановить с известного lockfile и пересобрать → закрыть дыру правилом.

---

10. Поддерживаемость человеком

Две документации, один факт — один источник [11 R1]: для людей `README.md` (первая ссылка «Если прод упал»), `ARCHITECTURE.md` ≤ 300 строк (codemap, инварианты, сквозной путь запроса), `apps/<id>/README.md` с таблицей «хочу найти / где / как проверить», `docs/runbooks/`, `docs/adr/`; для ИИ `CLAUDE.md`-дерево, rules, skills. Язык (решение пользователя): все доки на английском; у главного README русская копия `README.ru.md` (актуальность проверяет `tools/checks`); русский только для ответов агента. Runbooks (≤ экран, прогон без ИИ раз в квартал) [11 R3]: EMERGENCY, deploy, rollback (только сменой образа), hotfix, dependency-broken, api-down (рубильник `maintenance`), white-screen, config-env, secrets-leak. Пределы магии [11 R5]: нет DI и шины событий; кодген только OpenAPI, иконки, токены, с коммитом результата; ≤ 4 прыжка от страницы до сети; `max-lines` без ADR не отключается. Встроенная отладка [11 R7], [15]: `/version.json` = Sentry release = тег образа; ErrorBoundary на маршрут с «Детали для поддержки» и request id; `log.*` и `?debug=1`; рубильники в `/config.json` с владельцем и `expires`; текст ошибки эндпоинта ищется по коду в `mappings/*.json` и `apiErrors`. Чек-лист ревью ИИ-кода [11 R8] встроен в `rules-reviewer`; стартовые 8–12 ADR фиксируют принятые решения, включая «одно Next-приложение на проект».

---

11. Порядок работ

Все шесть шаблонов в объёме; волны задают порядок, не отрезают функции. Слой ошибок, фатальные экраны, utils и Storybook идут в ядро: без них ни один шаблон с API не соответствует правилам пользователя. Game идёт в волну 2 после landing, blog и site/base; shop — самый тяжёлый и изолированный, волна 3 [14 §6], [16 §9]. Итоговый README (английский + `README.ru.md`) пишется последним deliverable после кода, блоками по [20 §2].

Волна	Deliverables	Критерий выхода
0. Фундамент	проверка 12 механизмов Claude Code на реальной установке [08 §7]; smoke `openapi-fetch` + свой генератор фабрик по спеке реального бэкенда; smoke стилевого слоя по 9 рискам [13 R6]; smoke `site` (kit + `tv()` + MobX + Query-гидрация + next-i18next), бюджет JS, View Transitions на iOS / Android WebView / Telegram, CSP статики [16 §2.4]; Storybook + `addon-mcp` в Claude Code; ADR 0001–0012	таблица «механизм → работает/обход»; стилевой слой подтверждён или включён fallback; бюджеты site измерены

Волна	Deliverables	Критерий выхода
1. Ядро + tma + admin + AI + init + utils	ядро `packages/*` (без `lottie`, `realtime`, `astro-shared`): `MobxQuery v2`, `ScreenTransition`, полный pipeline ошибок (middleware с тестом порядка, resolution, `PageQueryGuard`, `appStatus` + `FailureScreen`, JSON маппинга + генератор, `apiErrors`), `devtools-core` с CLI `errors:*` и `i18n:*` (`--json`, флаг `draft`), `i18n/tools`; Storybook в `packages/ui` с MCP и Matrix у всех примитивов; `apps/utils` v1 без Screens; `tools/checks` и гейт; `.claude/*` (CLAUDE.md-дерево, rules включая API-rule, хуки с push-маркером, core skills включая `onboard`, `errors-map` и `speed`, агенты); `apps/tma` с `?perf=1`, демо-режимом и `pnpm tunnel`, `apps/admin`; Dockerfile на приложение; GitHub Actions; init с манифестом, бэкапом, dry-run, матрица для 2 шаблонов; runbooks EMERGENCY/deploy/rollback/hotfix; `ARCHITECTURE.md`, рабочий README	гейт зелёный, включая `errors:check` и `catalog:coverage`; init любого подмножества {tma, admin} даёт зелёный гейт и `pnpm dev`; `/onboard` даёт верную сводку; e2e: тост на 4xx, модалка через guard, NET-тост через `onError`, 403 banned → неубираемый экран и 503 от gate, 401 по старому токену не разлогинивает; `/errors-map` на новой операции спеки пишет маппинг и `draft`-тексты на всех локалях с зелёным гейтом и бейджем в utils; правка текста в utils даёт корректный diff; `/speed econom` ограничивает агентов по правилам; переходы и kit проверены на iPhone, Android, macOS, Windows; человек проходит rollback на staging без ИИ
2. landing + blog + site/base + game + anti-slop	`packages/astro-shared`, `apps/landing`, `apps/blog`; `apps/site` ядро с `auth-bff`, `i18n-routing`, `PageTransition`, двумя CSP-профилями, тем же маппингом в серверном fetch-слое, skills `site-*`; `packages/lottie` и `apps/game` с примерами wheel/reel/cards, очередью презентации, CSP-профилем, skills `game-*`; anti-slop lint на error, `lint:copy`, скрипт палитры; `design-reviewer`; Lighthouse CI; вкладка Screens и опция `crowdin`; runbooks dependency-broken/api-down/white-screen; матрица на 6 шаблонов	kit собирается в островах без провайдеров; CWV-бюджеты проходят; site: SSR-HTML с метаданными, защищённые маршруты редиректят, серверные ошибки попадают в error boundary по маппингу, переходы работают на iOS/Android; на реальном iPhone и среднем Android по `?perf=1` колесо/лента и Lottie-сцена держат 60 fps на прод-сборке, CSP без откатов; матрица одиночных и пар зелёная
3. site/shop + опции + модули	подшаблон `shop` (домен commerce, адаптеры с ошибками через общий маппинг, фасеты, `PaymentStep`, вебхук ревалидации, compose с Medusa для dev); опции `realtime`, `video`, `multi-instance`, `lottie`, `utils-deploy`; модуль `figma` (плагин-экспортёр, measure/diff, skills, инспектор); `design:scan`; CI-ревьюер; команда `add`; GitLab CI; итоговый README + `README.ru.md` по блокам [20 §2]	e2e commerce на `mock` до оплаты; образ site в обоих подшаблонах; `figma:measure` зелёный на реальном экране; nightly-матрица зелёная; новичок начинает работу по README без вопросов
Roadmap	`game-canvas`: отдельный шаблон на PixiJS [14 R10]; новые подшаблоны `site` (курсы, медиа); второе Next-приложение через `add`	решения после отчётов

---

12. Решения

Решения пользователя (2026-10-14/15): несколько шаблонов в одном монорепо; только фронтенд; Docker на своих серверах; только веб; Codex не приоритет; landing и blog на Astro, Next в `site`; shop структурно надмножество site, поэтому `site` с подшаблонами `base` | `shop`, ровно одно Next-приложение на проект (второе — через `add` в roadmap, «Deliberate do not fix»); MobX без DI + TanStack Query, без Zustand/Redux, React Compiler выключен; комментарии только там, где без них код не понять; `types.ts` / `consts.ts` / `utils.ts` и `Name.styled.ts` всегда отдельно, Tailwind v4 через `tv()`; GitHub первый; blog обязателен; game без canvas-движка, canvas-игры на PixiJS — будущий шаблон, `packages/lottie` обязателен только для game; тема TMA из `themeParams` с брендовым слоем; auth админки PKCE с порогом к BFF; Figma view-only, экспортёр запускает пользователь после запроса edit-доступа; shop — оба рынка через адаптеры; все доки на английском, у README русская копия, русский только в ответах; «оркестратор» только по запросу; `/onboard` после init; при готовом дизайне инициатива не поощряется; плавные переходы в каждом шаблоне; все устройства и способы ввода; маппинг ошибок и фатальные экраны ровно как в xRocket везде, где фронт зовёт бэкенд; utils-сайт на логике xrocket-tools, локальный, пишет в репо, не коммитит, `utils-deploy` поздней опцией, Crowdin опцией по умолчанию выкл.; агенты знают про utils и сами ведут маппинг ошибок и их тексты через CLI и `/errors-map`, эндпоинт без маппинга и текстов не готов, тексты агента помечаются как черновики и утверждаются пользователем, тексты пользователя всегда побеждают; каталог компонентов у каждого проекта крупнее лендинга и как шаг 1 дизайна с утверждением; режимы скорости `comfort` (по умолчанию) / `econom` / `business` через skill `speed` [20 §1]; итоговый README пишется после кода блоками [20 §2].

Решения оркестратора отмечены по тексту (§2–§11) со ссылками на отчёты; оспорить можно любое. Ключевые: iOS 16.4; `@medusajs/js-sdk`; Stripe Payment Element + ЮKassa redirect; у site образ на окружение; сокет напрямую к бэкенду по одноразовому тикету [17]; `openapi-fetch` вместо Orval, маппинг JSON в репо, тексты в `apiErrors`, исправления поломок xRocket [18]; один Storybook в `packages/ui` с Matrix-stories как вкладка Catalog [19]. Открытых вопросов к пользователю нет.

---

13. Риски и проверки

#	Риск	Мера
1	Механизмы Claude Code не проверены руками (`paths` с brace-glob, `onFailure`, MCP через `.cmd`, guard на Windows с кириллицей, чтение режима `speed` агентами) [08 §7]; Storybook MCP в preview [19 §4]	волна 0: таблица «механизм → работает/обход»; `ai:doctor` требует минимальную версию; exact-пин Storybook, MCP только как ускоритель
2	`MobxQuery v2`: гонки гидрации и отписки (xRocket потратил 512 строк [02 F3]); в site поверх RSC-prefetch без двойного запроса	явный API, тесты на `QueryObserver`, эталон в rules, smoke в волне 0
3	`openapi-fetch` в maintenance mode [18 §5.1]; маппинг без покрытия даёт «немые» ошибки (xRocket: 73 из ~237 путей) [18 §4]; агентские черновики текстов уйдут в прод незамеченными; utils пишет в репо	вендоринг при остановке поддержки; `errors:check` в гейте; `draft` виден в utils и в отчёте, `errors:list --draft` в чек-листе релиза; одна библиотека валидации, атомарная запись, allowlist путей
4	Ожидаемые мажоры и молодые версии (Astro 8, Next 17, tma.js 4 RC, dotLottie 0.x, Medusa ломает API в минорах) [17]	catalog пинит последний патч старше 7 дней, exact для нестабильных; smoke-сборки; `--update latest` nightly как сигнал; миграции мажоров одним PR; js-sdk Medusa пинится на версию бэкенда
5	Стилевой слой [13 R6]: kit как исходник в Next/Turbopack, острова без дублирования CSS, FOUC темы в SSR	smoke-сборка в волне 0; при провале fallback CSS Modules
6	Lottie и iOS: нет 60 fps без Worker-конвейера; iOS отказывает `play()` вне жеста; CSP молча выключает воркер, видео, WASM; эмуляция лжёт о fps [15]	`packages/lottie` + `lottie:audit` + видео-пререндер; `play()` в тапе, откат с причиной в `?perf=1`; CSP-профиль + e2e; замер только на устройстве и прод-сборке
7	Модель кэша Next: `cookies()` в layout делает сайт динамическим, персональное в `'use cache'` утекает; nonce-CSP несовместим со статикой; View Transitions различаются в Safari/WebView [16 §4, §6]	правила `apps/site/CLAUDE.md` с lint, skill `site-cache-review`, e2e «два пользователя — разные ответы»; два CSP-профиля, Report-Only, SRI в волне 0; CSS-fallback; бюджет site ≤ 130 KB gz
8	Модели копируют старый паттерн; правила без механизма не живут (xRocket: 233 отключения; Шахтёр: push вопреки тексту) [01], [15], [18]	lint на запрещённые импорты и паттерны; каждое правило с `enforced by` или `manual`; push только через `/push`-маркер
9	Объём (6 шаблонов с подшаблонами, utils, ~18 инструментов, 21+ skills), Medusa требует эксплуатации, дорогая матрица init, параллельные агенты ломают общую копию	волны с критериями выхода; один раннер проверок; `mock` для CI; PR только одиночные шаблоны, nightly pairwise; worktree на агента

Проверки на реальной установке сверх критериев §11: бюджеты JS каждого шаблона (tma ≤ 150, game ≤ 180, landing ≤ 50, site ≤ 130 KB gz); pipeline ошибок на реальном бэкенде; Figma-плагин на view-only и лимиты бесплатного плана; init-матрица на Windows с кириллическим путём; `/onboard` в свежей сессии.